

Building a Doctor Appointment Booking System with React, React Native, Node.js, and MongoDB

In today's fast-paced world, managing healthcare appointments efficiently is crucial. With the rise of telemedicine and digital solutions, creating a seamless doctor appointment booking system can significantly enhance patient experience. In this blog post, we'll explore how to build a doctor appointment booking application using React, React Native, Node.js, and MongoDB. We'll focus on key features like appointment booking, slot availability, and timely notifications.

Project Overview

Our application will enable users to:

- **Book Appointments:** Patients can select a doctor and book an available time slot.
- **Slot Availability:** The system will quickly show booked slots to avoid double bookings.
- **Managing Video Consultations:** With the rise of telemedicine, managing video consultations effectively is vital. Features to incorporate include.
- **Managing Prescriptions:** Providing patients access to their prescription history and detailed medication instructions.
- **Notifications:** Patients will receive reminders and notifications for their appointments, especially if any issues arise with the doctor.
- **Convenience:** Patients can book appointments from the comfort of their homes, at any time.
- **Efficiency:** Automated scheduling reduces administrative burdens on healthcare staff.

Architecture Overview

Frontend (React.js / React Native):

The user interface where patients can view doctors, available time slots, book appointments, and receive notifications.

Backend (Node.js):

RESTful APIs handle appointment booking, cancellations, time-slot management, and notifications.

Database (MongoDB):

Manages data related to doctors, patients, and appointments.

Notification System:

Scheduled reminders and real-time notifications via services like Firebase Cloud Messaging (FCM) or (OTS) for SMS.

Step-by-Step Guide to Building the App

1. Setting Up the Backend in Node.js

The backend serves as the brain of the application, handling everything from data storage to business logic.

Install Node.js and create the server:

```
npm init -y
```

```
npm install express mongoose body-parser
```

Create the Express server with routes for managing appointments, doctors, and patients.

```

4  const express = require('express');
5  const mongoose = require('mongoose');
6
7  const app = express();
8  app.use(express.json());
9
10 // Connect to MongoDB
11 mongoose.connect('mongodb://localhost/doctor-appointment', {
12   useNewUrlParser: true,
13   useUnifiedTopology: true
14 });
15
16 // Sample route for booking appointments
17 app.post('/appointments', (req, res) => {
18   // Appointment booking logic
19 });
20
21 app.listen(3000, () => {
22   console.log('Server running on port 3000');
23 });
24

```

Defining MongoDB Schema

Use Mongoose to define the database schema for doctors, patients, and appointments.

```

1  import mongoose from "mongoose";
2
3  const appointmentSchema = new mongoose.Schema({
4    doctorId: String,
5    patientId: String,
6    timeslot: Date,
7    status: String // Booked, Canceled, etc.
8  });
9
10 const Appointment = mongoose.model('Appointment', appointmentSchema);
11

```

3. Frontend: Booking Appointments (React & React Native)

Timeslot Details

Dr. Jagannath Mishra

Medicine

Cure Point

₹ 500

Select date for appointment

Mar 3rd 23
Friday

Mar 4th 23
Saturday

Mar 5th 23
Sunday

Mar 6th 23
Monday

Mar 7th 23
Tuesday

Mar 8th 23
Wednesday

Mar 9th 23
Thursday

Saturday

4th Mar 2023

Slots available for this day

Morning Slots

10:15 AM

10:30 AM

10:45 AM

11:00 AM

11:15 AM

11:30 AM

11:45 AM

The frontend will display available doctors and time slots, and allow users to book appointments.

In React and React Native, create a form to select the doctor and date/time slot.

Call the backend API to book an appointment.

On the backend, once an appointment is confirmed, update the MongoDB database and lock the slot.

```

53
54 const handleAppointmentBooking = async (doctorId, timeSlot) => {
55   const response = await fetch('http://localhost:3000/appointments', {
56     method: 'POST',
57     body: JSON.stringify({ doctorId, timeSlot }),
58     headers: { 'Content-Type': 'application/json' }
59   });
60   const data = await response.json();
61   if (data.success) {
62     alert('Appointment booked successfully!');
63   } else {
64     alert('Slot already booked');
65   }
66 };
67

```

4. Real-Time Slot Updates

To ensure real-time updates, use Socket.io or Polling.

Socket.io: Implement WebSockets to inform all users when a time slot is booked by someone else.

Polling: Set a regular interval to check for slot availability from the server.

5. Appointment Reminder Notifications

Use services like Twilio for SMS or Firebase Cloud Messaging for push notifications to remind patients about their appointments.

On the backend, create a scheduler to send notifications 10 minutes before the appointment.

Node-cron can be used to schedule reminders.

Real-time Slot Locking

To ensure that booked slots are instantly marked as unavailable, we can implement Socket.io for real-time communication or use polling to frequently update available slots.

```
const io = require('socket.io')(server);

io.on('connection', (socket) => {
  socket.on('booked-slot', (data) => {
    socket.broadcast.emit('update-slots', data);
  });
});
```

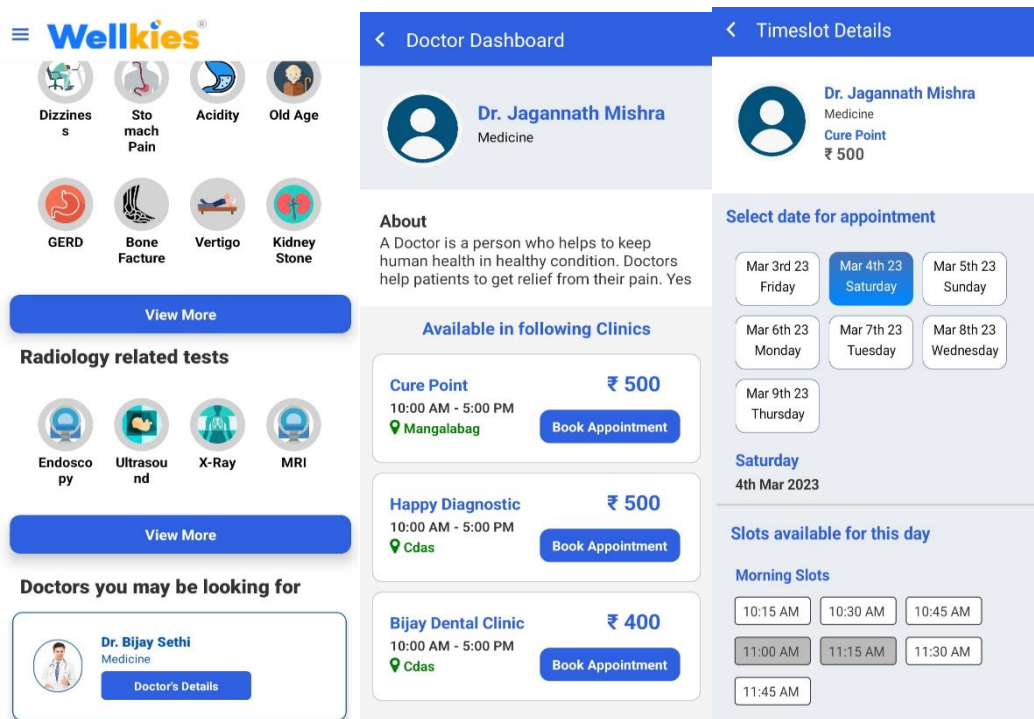
Doctor Availability Notifications

If a doctor becomes unavailable or encounters any issues, notify the patient instantly using backend API calls and the notification service.

Backend route to trigger notifications:

```
app.post('/notify-issue', (req, res) => {
  const { patientId, message } = req.body;
  // Send notification logic here
});
```

Here are some output images



Conclusion

Developing a Doctor Appointment Booking App using React, React Native, Node.js, and MongoDB ensures a seamless experience for patients and doctors. With features like

real-time slot booking, reminders, online payment, video consultation, and notifications, the app improves the efficiency of healthcare service delivery. This tech stack ensures the app is scalable, fast, and user-friendly, making it a valuable tool in modern healthcare.

By integrating features like real-time notifications and instant slot locking, you provide users with a smooth and reliable experience, ensuring they never miss an appointment.